

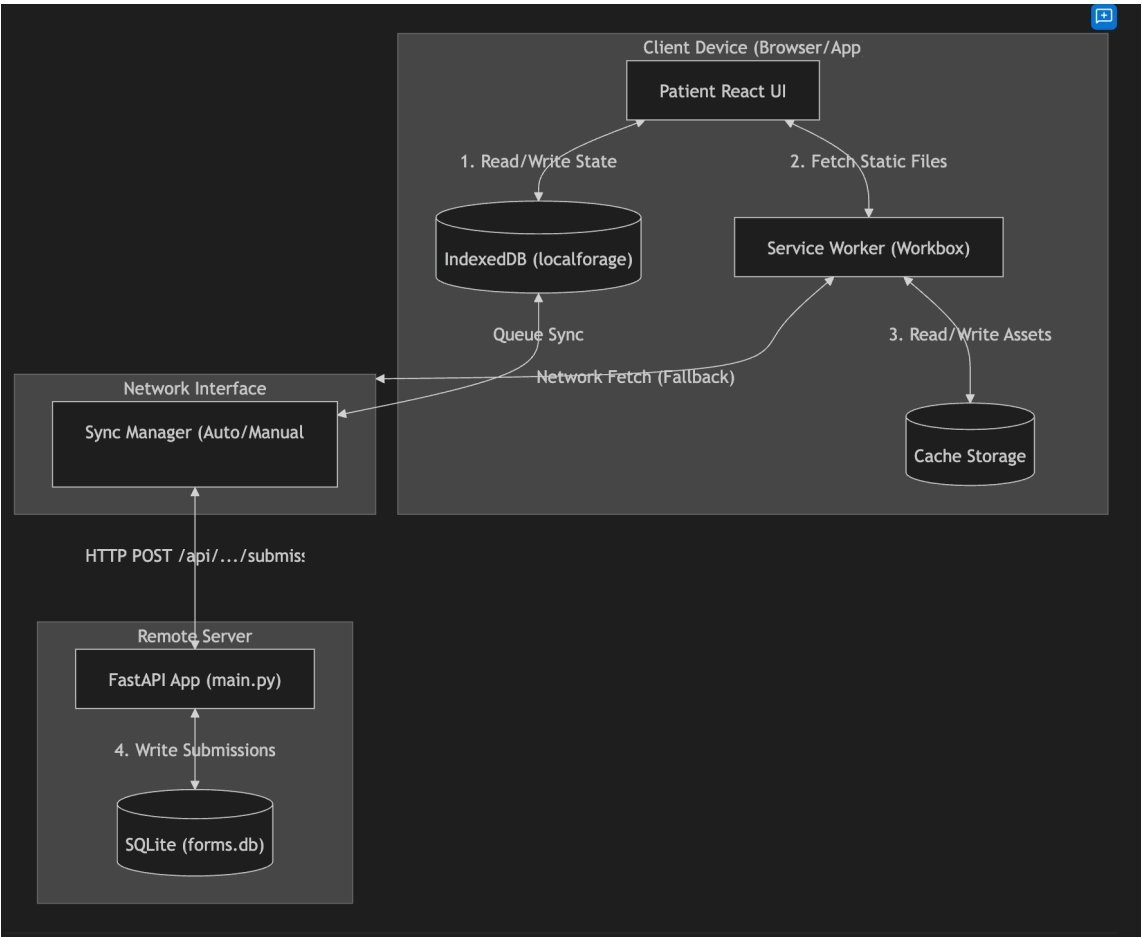
# HaloFormCraft: Offline-First Architecture & Sync Mechanism

This document details the architectural blueprint, technological selection, and state flow for the **Offline-First Capabilities** of the **HaloFormCraft Patient Portal ( patient-ui )**.

Our primary goal is to allow clinical patients to complete and sign forms in low-connectivity or completely offline environments (e.g., remote clinics, hospital basements) without losing progress or critical data (such as signatures and GPS coordinates).

## 1. Core Architecture Overview

To achieve an offline-first experience, we move away from a traditional client-server request model to a **Decoupled Cached-Storage Architecture**. The application treats local storage (IndexedDB) as its primary source of truth, lazily syncing with the remote backend (FastAPI) when the network is available.



## 2. Technology Stack & Utilities

To implement this architecture, we will use three standard, lightweight tools:

| Technology                                  | Purpose                     | Implementation Details                                                                                                                                                  |
|---------------------------------------------|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Vite PWA Plugin</b><br>(vite-plugin-pwa) | Asset compilation & caching | Integrates Workbox with Vite to automatically generate and register the Service Worker and configure the <code>manifest.json</code> .                                   |
| <b>Service Worker</b><br>(Workbox)          | Offline application loading | Intercepts HTTP requests. Implements a <b>Cache-First</b> strategy for static files (HTML, CSS, JS, Fonts) and <b>Network-First</b> for dynamic configuration files.    |
| <b>IndexedDB</b><br>(localforage)           | Local database engine       | Provides asynchronous storage (Map/Set style API backed by IndexedDB) to store heavy form schemas and queued submissions (including Base64 signatures and JSON arrays). |
| <b>Web App Manifest</b><br>(manifest.json)  | App Installability          | Enables "Add to Home Screen" behavior on Android/iOS/Desktop, launching the portal in a full-screen, native-like standalone window.                                     |

### 3. Storage Schema (IndexedDB)

Using `localforage`, we initialize three isolated storage instances:

```
// 1. Storage for available forms
const formCacheStore = localforage.createInstance({
  name: "HaloFormCraft",
  storeName: "cached_forms"
});

// 2. Queue for offline submissions
const submissionQueueStore = localforage.createInstance({
  name: "HaloFormCraft",
  storeName: "submission_queue"
});

// 3. User & Session tracking
const sessionStore = localforage.createInstance({
  name: "HaloFormCraft",
  storeName: "user_session"
});
```

#### Data Structures

##### Queued Submissions Store ( `submission_queue` )

Each offline submission is assigned a unique, client-side generated UUID and tracked with sync statuses:

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "formId": 4,
  "formTitle": "General Intake Form v1.0",
```

```
"payload": {
  "patient_name": "Jane Doe",
  "signature": "data:image/png;base64,iVBORw0KGgo...",
  "gps_coordinates": { "lat": 40.7128, "lng": -74.0060 },
  "answers": { "height": 170, "weight": 70, "bmi": 24.2 }
},
"timestamp": "2026-05-20T08:30:00Z",
"syncStatus": "PENDING",
"retryCount": 0,
"errorMessage": null
}
```

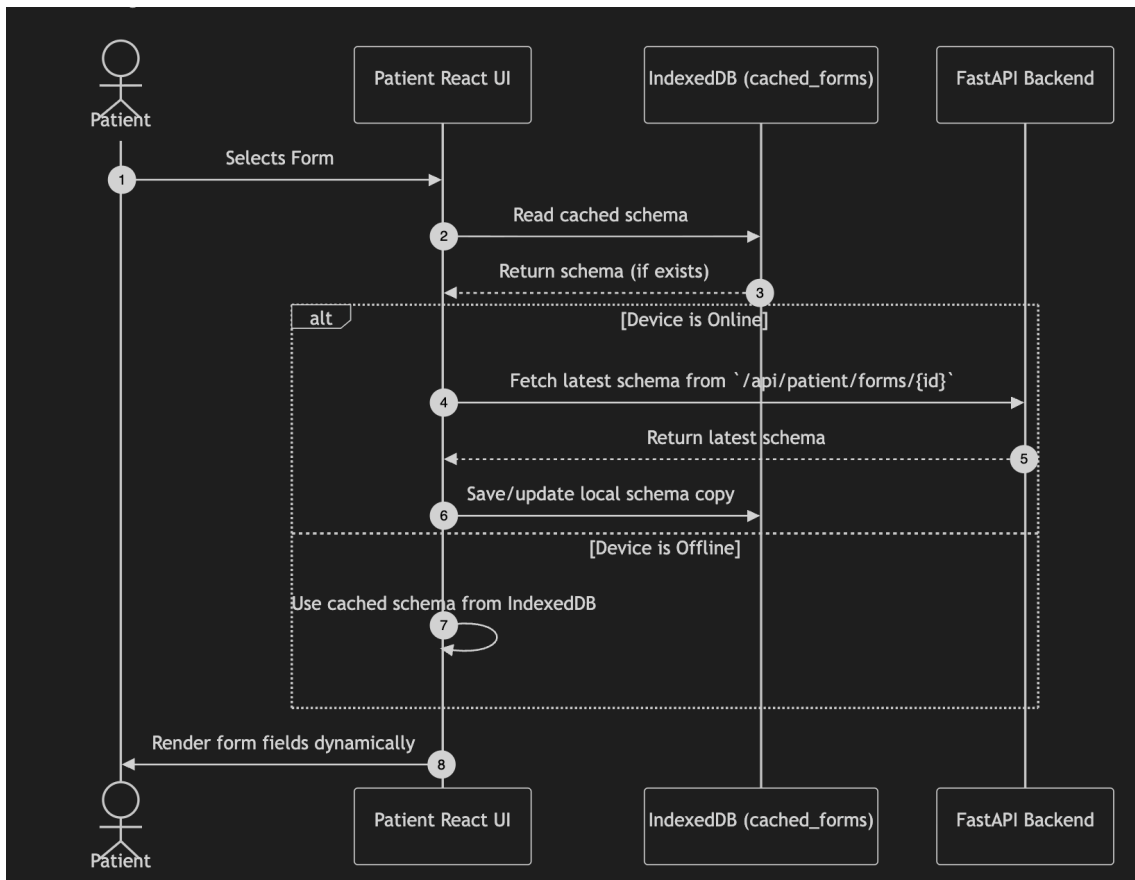
## 4. Detailed Step-by-Step Mechanisms

### A. Bootstrapping & Caching (The Service Worker Lifecycle)

1. **Compilation:** During building, `vite-plugin-pwa` calculates the hashes of all compiled files ( `index.html` , assets, etc.) and lists them in a precache manifest.
2. **Installation:** When the patient visits the portal online, the Service Worker installs and downloads all listed assets into the browser's Cache Storage.
3. **Activation:** The service worker takes control. The next time the user loads the app—**even with airplane mode turned on**—the Service Worker serves all assets directly from the Cache instantly.

### B. Form Loading (Schema Caching)

When a user navigates to a form:



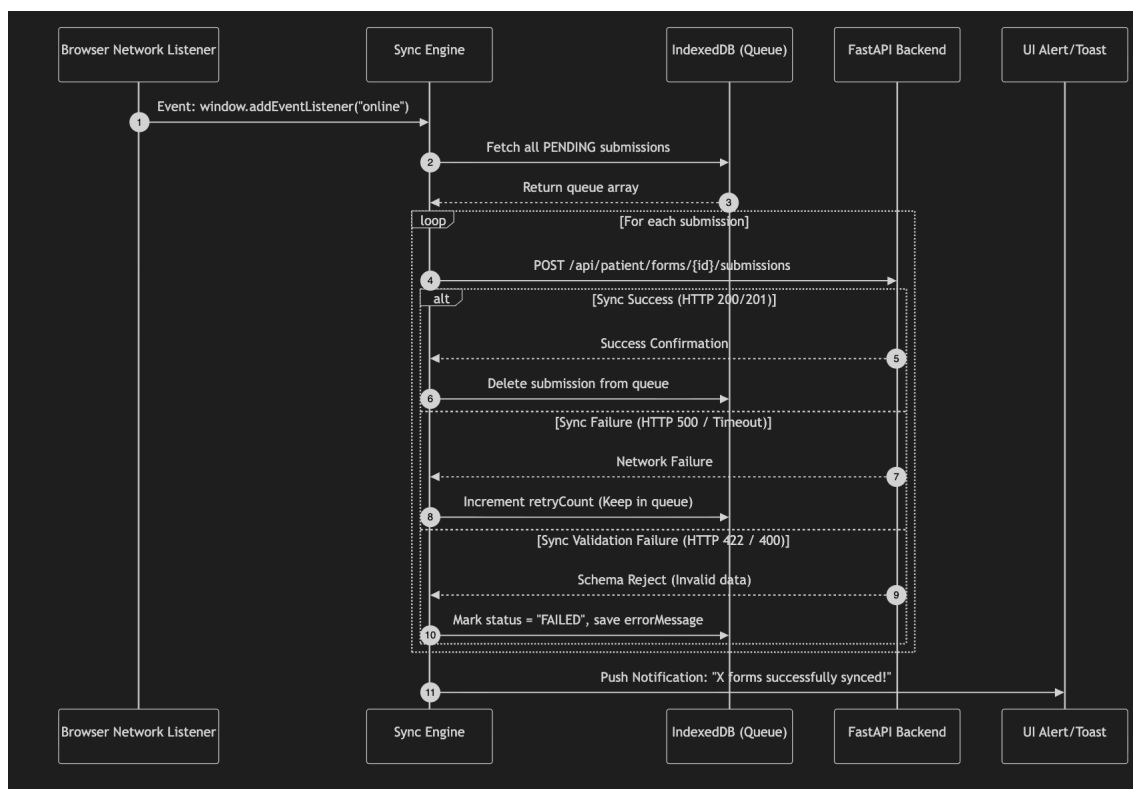
### C. Offline Form Submission (Queuing)

When the user clicks **Submit** while offline:

1. **Validation:** React Hook Form processes validation (ensures signatures are drawn, required numbers are correct).
2. **Connection Check:** The app checks the state of `navigator.onLine`.
3. **Offline Capture:** If offline, the submission payload is saved to `submission_queue` in IndexedDB with `syncStatus = "PENDING"`.
4. **User Feedback:** The UI shows a success message: *"Form completed successfully! Your answers are saved locally and will automatically sync when you reconnect."*

### D. Re-Synchronization Mechanism (Going back Online)

When connectivity is restored, the synchronization process triggers automatically:



## 5. User Interface (UI) Experience Elements

To ensure patients feel completely safe about their clinical data during offline mode, we will implement three vital visual indicators:

### 1. Network Status Indicator Banner:

- **Online:** Small, auto-dismissing green toast: *"Connected: You are online."*
- **Offline:** Sticky, high-contrast orange bar at the top of the viewport: **Offline Mode:** Your forms will be saved locally on this device and securely synced once you reconnect.

### 2. Pending Queue Count:

- A persistent badge on the home portal dashboard displaying the number of pending items: e.g., **3 Forms Pending Sync**.
- Includes a manual **"Sync Now"** button, allowing users to force a sync if they believe their internet is back but the browser's native API hasn't fired the event yet.

### 3. Inline Submission History:

- A simple screen showcasing a list of recent submissions and their state:
  - **Green Badge | Synced** (Sent to clinic databases)
  - **Orange Badge | Pending Sync** (Saved locally on device)
  - **Red Badge | Failed Sync** (Allows the user to view why, correct the details, and re-trigger sync)

## 6. Resolving Complex Sync Conflict Scenarios

| Scenario                                        | Risk                                                                                               | Mitigation Strategy                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Form Schema Updated Online while Offline</b> | Patient submits a form built on a schema that was deleted or heavily modified by an administrator. | Submissions include the <code>schema_version</code> or timestamp. The backend will parse the answers against the version schema stored in <code>form_versions</code> . If a critical mismatch occurs, the form is stored in a draft review queue for administrators rather than crashing or being lost. |
| <b>Concurrent Submissions</b>                   | Multiple users using the same tablet device offline.                                               | The queue is built on individual submission UUIDs rather than sequential numbers. Submissions are processed sequentially and verified separately without overlapping each other.                                                                                                                        |
| <b>Heavy Signature Files</b>                    | Base64 signatures bloating IndexedDB limit.                                                        | IndexedDB allows up to <b>80% of total disk space</b> in modern browsers (e.g., several gigabytes). A detailed form with base64 signatures rarely exceeds 150KB, meaning tablets can safely store thousands of submissions offline without memory issues.                                               |